

# Ikigai -- A Biologically Grounded Digital Organism: Development Report, Days 46–47

Hitoshi AI Labs -- NeuroSeed Project

**Author:** Prince Siddhpara, Founder -- Hitoshi AI Labs **Date:** May 11–12, 2026 (Day 46: May 11 | Day 47: May 12)

**Project:** NeuroSeed **Subject:** Days 46–47 -- Symbolic String Encoding, Replay Consolidation, and Structural Prediction

**Classification:** Research Document -- Computational Neuroscience / Artificial Life / Cognitive Systems Engineering

---

## 1. Objective

Day 45 closed the bidirectional task-to-neural feedback loop: task performance statistics produced a signal that fed back into neural scoring and task-graph branching. The coupling was narrow, deterministic, bounded, and not yet dominant -- but structurally real. The neural layer was now participating in a closed-loop relationship with a symbolic task signal for the first time. The architecture was complete at the neural-task interface; the remaining gap was the absence of symbolic representation. The agent could detect transitions, count them, and react to them -- but it had no internal structure for representing what those transitions were about. There was no symbolic encoding layer.

Days 46–47 address this gap by introducing a full symbolic representation and learning stack operating at two complementary granularities. Day 46 constructs a token-level symbolic encoding system: identifiers, operators, and punctuation from code-like strings are tokenized, mapped to deterministic hypervectors via MD5-seeded RNG, composed into sequence representations via XOR-binding and majority-vote bundling, and evaluated against a held-out curriculum. Token replay runs during sleep, measurably reducing held-out prediction error across consolidation cycles. Day 47 lifts the same architecture to the raw string level: entire strings are encoded into stable order-sensitive HVs, curriculum learning and sleep replay are applied at string granularity, cross-modal coupling between token and string representations is measured, and multi-cycle consolidation accumulates across six sleep cycles. The final three packs of Day 47 probe the limits of generalization: a tier-2 vocabulary expansion test, a character-level bigram predictor, and the first predictive VSA binding operator -- a structural transformation that captures the relationship between string classes in hypervector space.

The cumulative result across Days 46–47 is 275+ new assertions across 22 packs, all PASS. All OFF paths are bit-identical. Zero routing or TC drift is confirmed for every pack. No randomness was introduced at any stage.

---

## 2. Day 46 -- Token-Level Symbolic Learning and Replay

### Pack 1 -- Deterministic Token Encoder (default OFF, token hypervector foundation)

Introduces a full token encoding pipeline operating below the string level. A `tokenize(text)` method uses a regex pattern to segment code-like strings into identifiers, integers, two-character operators, and punctuation tokens. Each unique token is assigned a stable 400-bit hypervector via an MD5-seeded local RNG: the global random state is never touched, so the encoding is fully deterministic and isolated from any run-to-run variation. Adjacent token pairs are composed via XOR binding to produce positional pair vectors; these are bundled via majority vote across the full sequence to produce a single 400-bit sequence representation (`token_seq_vec`). A parallel transition predictor tracks bigrams in token space: `token_transition_counts[prev_tok][next_tok]` is updated each sequence update; `token_transition_probs` is rebuilt from counts every 50 sequence updates. Token-level prediction error (`token_te`) is computed as 1.0 minus the probability assigned to the actual next token. The probe string "if x == 1 : return x" contains 6 unique tokens. Pack 1 is injected before `time.sleep(0.010)` and fires only on waking ticks. Bit-identical routing/TC/TE confirmed against the Day 45 baseline when disabled.

### Pack 2 -- Code Token Curriculum with Held-Out Generalization (default OFF, three-phase symbolic curriculum)

Introduces a deterministic three-phase curriculum operating over the token encoder from Pack 1. Phase 0 presents the seen string S0 ("if x == 1 : return x"); Phase 1 presents the held-out string H ("if x == 0 : x = x + 1"); Phase 2 presents the seen string S1 ("while x == 0 : x = x + 1"). The curriculum cycles through phases by index: `phase = token_seq_updates % 3`. Separate TE accumulators for seen and held-out phases allow per-phase error tracking. The structural ambiguity is intentional: the held-out string H shares the last-seen pair "=>1" with S0 but conflicts with S1's "=>x" at the same context "". After training on both seen strings, the predictor assigns roughly equal probability to "1" and "x" at the "" context, producing persistent held-out TE  $\approx 0.67$ . Seen TE drops to  $\approx 0.035$  as dominant transitions are memorized. The held-out vs seen separation becomes stable within the first curriculum pass and is maintained thereafter.

### Pack 3 -- Token-State Behavioral Coupling (default OFF, lexical density-to-behavior bridge)

Computes a scalar behavioral signal from token composition properties of the current string: `ctrl_density` is the fraction of tokens that are control keywords (if, while, for, return); `math_density` is the fraction that are operators (==, +, -, \*, /). The behavioral signal is  $0.5 \times \text{ctrl\_density} - 0.25 \times \text{math\_density}$ , producing three distinct values across the curriculum strings: -0.025 (math-heavy), 0.0 (neutral), +0.107 (control-heavy). An EMA with  $\alpha = 0.1$  smooths this signal into `token_behavior_ema`. At BG scoring time, the EMA modulates approach, wait, and explore scores: `approach/wait += 0.5 \times K\_TOKEN\_BEHAVIOR \times ema`; `explore -= 0.5 \times K\_TOKEN\_BEHAVIOR \times ema`. The gate is `vsa_sw_timer > 0`, producing a 1-tick lag. `K_TOKEN_BEHAVIOR = 0.004` is at the same level as `K_OUTCOME_BIAS`, consistent with the K ordering established in Day 43. Delta across all action channels remains below 2%. Zero action-winner changes result.

### Pack 4 -- Token-Conditioned Task Feedback (default OFF, double-EMA coupling to task state)

Extends the token-state coupling to incorporate task performance feedback. A second EMA `token_task_feedback_ema` is computed as  $0.9 \times \text{prev} + 0.1 \times \text{token\_behavior\_ema}$ , creating a 2-tick lag relative to the raw lexical signal. This double-EMA is applied at BG scoring time as an extra score contribution  $K\_TOKEN\_TASK\_FB \times \text{task\_feedback\_ema} \times \text{token\_task\_feedback\_ema}$ , added after the Pack 3 apply block. The gate requires both `enable_task_feedback` and `vsa_sw_timer > 0`.  $K\_TOKEN\_TASK\_FB = 0.003$  is strictly below  $K\_TOKEN\_BEHAVIOR = 0.004$ . The extra contribution in the validation run is approximately  $2.84 \times 10^{-5}$  per tick - never sufficient to flip any action winner. Pack 4 demonstrates that a task-aware lexical signal can be routed to action scoring without disturbing the existing behavioral hierarchy.

#### **Pack 5 -- Token Replay Consolidation (default OFF, sleep-gated held-out rehearsal)**

Introduces the first sleep-gated symbolic replay mechanism. During sleep ticks (when `sleeping == True`), Pack 5 fires in complement to Packs 1–4 (which fire only when awake). Priority logic: if `heldout_avg_te > seen_avg_te`, the held-out sequence is selected for replay; otherwise, S1 (the more recently seen sequence) is replayed. Replay calls `token_process_seq` and `token_encode_seq` from the Pack 1 infrastructure -- no new predictor state is created. The effect is to reinforce held-out token transitions during offline periods, counteracting the distributional dominance of seen sequences during waking. In the 2,000-tick validation run, 53 sleep ticks occur; all 53 replay in held-out mode (`heldout_avg_te ≈ 0.498 > seen_avg_te ≈ 0.035`). Replay TE  $\approx 0.498$  per replay tick, reflecting the predictor's genuine uncertainty about the held-out string under the ambiguous "==" context.

#### **Pack 6 -- Replay Retention Measurement (default OFF, pre/post sleep TE snapshots)**

Measures the effect of sleep replay on held-out prediction quality using a circular pre-window buffer and a post-window capture gate. The pre-window records the last `window_len` waking ticks before sleep onset (using a circular buffer committed at the onset transition); the post-window captures the first `window_len` waking ticks after sleep ends, gated by a flag `_rr_had_sleep` that is set at first sleep onset. `rr_te_gap = post_avg_te - pre_avg_te`: negative means post-sleep TE is lower (improvement). In the 2,000-tick validation run with 3 sleep cycles: pre = 24 ticks, post = 24 ticks; `te_gap = -0.0053`. The specific improvement mechanism: 53 sleep replay events shift  $P(1|==)$  from 0.500 to 0.527, lowering held-out TE from 0.4994 to 0.4727. The magnitude is small but directionally consistent -- sleep replay provides measurable consolidation at the token level.

#### **Pack 7 -- Code Completion Benchmark (default OFF, masking-based structural probe)**

Constructs a read-only benchmark over the existing token transition probabilities from Pack 1. Seen probe strings use tokens with dominant transitions ( $P > 0.95$  after training); held-out probe strings use tokens at the == context where probabilities are split between two continuations. No new training data is provided: Pack 7 reads from the existing `token_transition_probs` dictionary and reports TE at the probe boundary. Seen masking reduces TE to 0.0345 (dominant transitions reliably predicted); held-out uncertainty TE = 1.0000 (equal probability between two valid continuations, producing worst-case error). The benchmark confirms that the predictor's confidence is structurally meaningful: high confidence corresponds to structurally unambiguous transitions; low confidence corresponds to genuine lexical ambiguity. No `ikigai.py` changes are required; the benchmark is computed entirely in the experiment wrapper.

#### **Pack 8 -- In-Vocabulary Structural Probe (default OFF, difficulty graded by ambiguity depth)**

Replaces the OOV failure mode from earlier structural probes with a difficulty measure based on ambiguity depth within the trained vocabulary. All probe tokens are present in `token_transition_probs`; held-out difficulty comes from structural conflict (multiple equally valid continuations at the same context) rather than vocabulary absence. Probe strings are graded across three difficulty tiers: easy (dominant single continuation,  $P \approx 0.95$ ), medium (two plausible continuations,  $P \approx 0.50$  split), and hard (three-way split,  $P \approx 0.33$  per continuation). Easy tier TE = 0.041; medium tier TE = 0.678; hard tier TE = 0.999. `te_gap` between easy and hard = 0.958. The probe confirms that structural difficulty, not vocabulary coverage, is the binding constraint on generalization in the current architecture. 76/76 PASS across Day 46 Packs 1–8.

---

## 3. Day 47 -- String-Level VSA Structure and Predictive Binding Operators

### Pack 1 -- Deterministic String-to-Hypervector Encoder (default OFF, raw string HV foundation)

Constructs a stable 400-bit hypervector from any raw string without tokenization. The encoding procedure uses two complementary components: position-bound character HVs (each character's HV is XOR-shifted by its position index) and character bigram HVs (adjacent character pairs encoded as XOR of the two character HVs, bundled across all bigram positions). The final HV is the majority vote of all position-bound and bigram contributions. `string_encode(s)` is deterministic, requires no training, and is stable across runs. Two structural properties are verified empirically: (1) `sim(full_encoding, bag_of_chars)  $\approx 0.485$` , confirming that position and bigram contributions break pure bag-of-characters equivalence -- order matters; (2) `sim("if x", "x if")  $\approx 0.585$` , confirming that character-level overlap is preserved but ordering produces measurable divergence. The encoder is injected into `ikigai.py` alongside the `string_encode` method added to the `IkigaiContext` class in `state.py`. All downstream packs depend on this encoder. Zero routing/TC/TE drift confirmed when disabled.

### Pack 2 -- String Curriculum with Held-Out Generalization (default OFF, string-level curriculum learning)

Applies the same three-phase curriculum structure from Day 46 Pack 2 to the string-level encoder. Three seen strings (`S0 = "if x == 1 : return x"`, `S1 = "while x == 0 : x = x + 1"`, `S2 = "if x == 0 : return x"`) and one held-out string (`H = "while x == 1 : x = x + 1"`) are cycled through a 10-phase schedule [`S0 S1 S2 S0 S1 S2 S0 S1 S2 H`]. The whole-string predictor `string_transition_counts[prev_string][next_string]` is keyed by exact string identity; each tick provides one training sample (`prev  $\rightarrow$  curr` string transition). Seen TE drops to  $\approx 0.035$  as dominant transitions are memorized; held-out TE remains at  $\approx 0.67$ . An additional structural factor is present: Pack 1 of the day-level experiment uses a 5-string cycling baseline that introduces additional `S2  $\rightarrow$  non-H` transitions during the curriculum warm-up phase, diluting `P(H|S2)` from the theoretical 0.33 toward  $\approx 0.20$ . This dilution is a permanent feature of the architecture at this string count and does not resolve within the 1,500-tick window.

### Pack 3 -- Cross-Modal String-Token Alignment (default OFF, inter-encoder similarity measurement)

Computes the cosine similarity between the string-level HV from Pack 1 and the token-level sequence HV from Day 46 Pack 1 for the same input string. The two encoders are structurally independent: the string encoder operates on raw characters with positional binding; the token encoder operates on lexically segmented units with XOR-bind adjacent

pairs. Mean alignment  $\text{sim} \approx 0.51$  (near random-chance for 400-bit binary vectors). The near-zero alignment is the expected result: two independently initialized encoding systems applied to the same input produce largely orthogonal representations. Pack 3 confirms that the string and token layers are operating in complementary rather than redundant representational spaces. No `ikigai.py` changes are required; alignment is computed in the experiment wrapper at each tick. Observation-only, zero drift.

#### **Pack 4 -- String-Level Held-Out Replay (default OFF, sleep-gated string sequence rehearsal)**

Introduces sleep-gated replay at the string level, mirroring Day 46 Pack 5. During sleep ticks, if `string_curriculum_heldout_te > string_curriculum_seen_te`, the sequence [S2, H] is replayed: S2 is presented as the previous string and H as the current, updating the `string_transition_counts` predictor toward  $P(H|S2)$ . The replay mechanism uses the same `string_process_seq` infrastructure that waking curriculum ticks use; no separate predictor or encoding pathway is created. In the 1,500-tick validation run with one sleep cycle (53 sleep ticks), all 53 ticks replay in held-out mode.  $P(H|S2)$  increases from 0.1994 to 0.2546 across the single consolidation cycle, a gain of +0.055. The argmax threshold is not crossed at this run length: the predictor does not flip to predicting H as the most likely successor to S2. The improvement is real and measurable in the probability distribution before it becomes observable in the argmax.

#### **Pack 5 -- String Replay Retention (default OFF, pre/post sleep TE measurement at string level)**

Applies the same pre/post sleep snapshot methodology from Day 46 Pack 6 to string-level TE. The pre-window captures held-out TE for the last `window_len` waking ticks before sleep onset; the post-window captures held-out TE for the first `window_len` waking ticks after sleep ends. `te_gap = post_avg_te - pre_avg_te`. In the 1,500-tick validation run: `te_gap = -0.0613` (post-sleep held-out TE improves by 0.06 relative to pre-sleep). The magnitude is larger than the token-level improvement from Day 46 Pack 6 (`te_gap = -0.005`) because the string-level predictor has fewer training samples per key -- a single sleep cycle of 53 replay events produces a proportionally larger shift in the `string_transition_counts` distribution than the same number of events does in the token predictor, which sees more training signal per key per waking tick.

#### **Pack 6 -- Cross-Modal TE Coupling (default OFF, running Pearson correlation between string and token TE)**

Computes the running Pearson correlation between string-level TE and token-level TE across waking ticks. `cmt_pearson_r` is updated using Welford's online algorithm applied to the two-variable case: running means, variances, and covariance are maintained without storing the full tick history. In the 1,447-tick validation run,  $r = 0.2321$ . The correlation is positive but below the 0.3 milestone: string and token prediction errors co-vary weakly because both encounter high uncertainty at similar points in the curriculum cycle (when the held-out string or held-out token bigrams appear), but the two curricula are not identical and drive distinct error distributions. The below-0.3 result confirms that the two encoders are genuinely complementary -- correlated enough to share a common error regime, orthogonal enough to provide independent information. Observation-only. Zero drift.

#### **Pack 7 -- Multi-Cycle Sleep Accumulation (default OFF, cross-cycle replay accounting and $P(H|S2)$ trend tracking)**

Extends the single-cycle replay measurement from Pack 5 to accumulate statistics across all sleep cycles in a 5,000-tick run. On each wake-to-sleep transition: the current  $P(H|S2)$  snapshot is taken as `ph_start` (first cycle only); the Pack 4 replay counters are snapshotted as the cycle baseline. On each sleep-to-wake transition: the per-cycle replay delta (total

replays, correct predictions, average TE) is accumulated into running totals; `ph_end` and `ph_trend` are updated from the current held-out accuracy. In the 5,000-tick run with 6 sleep cycles, 312 replay events accumulate across all cycles at `avg_replay_te = 0.7273`. `ph_trend = ph_end - ph_start = 0.0`: the held-out accuracy (`string_curriculum_acc_heldout`) remains 0.0 throughout all six cycles. The result is structurally correct, not a bug: Pack 1's 5-string cycling introduces diluting  $S_2 \rightarrow$  non-H transitions that keep  $P(H|S_2) \approx 0.25$  below the argmax flip threshold. The predictor's probability estimate for  $H|S_2$  is improving, but never crosses the winner boundary within this run length. Replay events are genuine; the argmax is simply not moving.

#### **Pack 8 -- String Vocabulary Expansion (default OFF, tier-2 novel string curriculum with transfer measurement)**

Introduces three tier-2 strings ( $T_0 = \text{"while } x == 1 : \text{return } x"$ ,  $T_1 = \text{"if } x == 0 : x = x + 1"$ ,  $T_2 = \text{"if } x == 1 : x = x + 1"$ ) and one tier-2 held-out string ( $T_{\text{HELD}} = \text{"while } x == 0 : \text{return } x"$ ) using only characters present in the tier-1 vocabulary. The tier-2 curriculum activates after `string_curriculum_idx >= 60` and cycles through a 10-phase schedule identical in structure to Pack 2. Transfer is measured by computing HV structural similarity between corresponding tier-1 and tier-2 strings at `tier2_idx == 1`: `sim(T0, S0) = 0.68`; `sim(T1, S1) = 0.79`. Both similarities exceed 0.5, confirming that the string encoder captures structural similarity between strings that differ only in keyword substitution (if  $\leftrightarrow$  while) while preserving the remaining character structure. The whole-string predictor transfers zero information: tier-2 strings have never been seen as keys or values in `string_transition_counts`, so the cold start is identical to any unseen string. Transfer ratio  $\approx 1.0$  (warm and cold TE converge at the same rate). The encoder knows these strings are similar; the predictor does not.

#### **Pack 9 -- Tier-2 Transfer Probe (default OFF, interleaved tier-1/tier-2 transfer measurement)**

Probes whole-string transfer by interleaving tier-1 strings ( $S_0, S_1, S_2$ ) with tier-2 strings ( $T_0, T_1, T_2, T_{\text{HELD}}$ ) in a controlled schedule and measuring whether prior tier-1 training helps the predictor learn tier-2 faster. Two sub-experiments are run: (1) a warm run where the predictor has already been trained on tier-1 sequences before encountering tier-2 strings -- the schedule uses `_st2_sub = (idx // 4) % 3` for both tier-1 and tier-2 to ensure unambiguous 1:1 pairing ( $S_i$  always appears in context for  $T_i$ ); (2) a cold run where only tier-2 strings are presented from the start, using a 4-cycle schedule that includes  $T_{\text{HELD}}$ . Empirically: `warm_te = 0.754`, `cold_te = 0.230`, `transfer_ratio = warm_te / cold_te = 0.305`. Cold run (pure tier-2 exposure) converges approximately 3× faster than the warm run. The warm run's higher TE reflects interference from the established tier-1 transition distribution -- the shared predictor must overwrite tier-1 associations with tier-2 associations, producing elevated error during the transition period. No structural generalization from tier-1 to tier-2 is present at the whole-string key level.

#### **Pack 10 -- Character-Level Transition Predictor (default OFF, character bigram generalization across tier boundary)**

Introduces a character-level predictor that operates on individual characters within strings rather than on whole strings as atomic keys. `char_transition_counts[prev_char][next_char]` is updated for each adjacent character pair within the current string. `char_transition_probs` is rebuilt every 50 sequence updates. Character-level TE is computed BEFORE updating counts (true surprise measure): for each adjacent pair, the probability assigned to the actual next character is subtracted from 1.0. This ensures that TE reflects genuine prediction error rather than post-hoc training. In the transfer probe: `warm_te = 0.4358`, `cold_te = 0.4363`, `transfer_ratio = 1.001`. The near-unity ratio indicates marginal positive transfer: tier-1 character bigrams ( $x \rightarrow \text{space}$ ,  $\text{space} \rightarrow ==$ ,  $== \rightarrow \text{space}$ , etc.) are largely shared with tier-2 strings (which differ only in keyword substitutions: "if"  $\leftrightarrow$  "while"), providing genuine but small prior knowledge. The

advantage is real but saturates quickly -- both warm and cold runs converge to the same char bigram distribution within a few dozen string presentations. 17 unique character transition keys are maintained. No `ikigai.py` changes beyond the existing Pack 9 injection point.

#### **Pack 11 -- Syntax-Level VSA Binding Predictor (default OFF, structural HV transformation learning)**

Introduces the first predictive hypervector operator in the architecture. The binding operator approach treats the transition from one string HV to the next as a fixed structural transformation that can be learned and applied predictively. For each consecutive pair of string presentations (`prev_hv`, `curr_hv`), the XOR `vsa_bind(prev_hv, curr_hv)` is accumulated into a running bit-sum `_vsp_op_counts[i]` (count of 1s at each of 400 bit positions, across all observed transitions).

Every 10 transitions, the `vsa_binding_op` is reconstructed as a majority-vote hypervector: bit  $i = 1$  if `_vsp_op_counts[i] > total/2`. Prediction is then `vsa_bind(curr_hv, vsa_binding_op)`, and prediction quality is `vsa_similarity(predicted, next_hv)`. The accumulator uses only 400 integers and one count -- memory is bounded regardless of run length. In the 1,500-tick validation run: `vsa_pred_sim_ema = 0.7167` after EMA smoothing with  $\alpha = 0.05$ . Seen sim = 0.7188, held-out sim = 0.7239. Both exceed 0.5 (random baseline for 400-bit binary vectors), with the held-out similarity marginally exceeding the seen similarity. The result confirms that the if  $\leftrightarrow$  while keyword substitution produces a consistent directional transformation in HV space: the binding operator captures this structural relationship and predicts it reliably. This is the first demonstration that the HV representations support structural prediction beyond transition memorization.

---

## 4. Architectural Interpretation

### **What the system can now do**

The architecture entering Day 48 possesses the following demonstrated capabilities:

- Encode raw strings into stable, order-sensitive 400-bit hypervectors using deterministic character-level composition (Day 47 Pack 1).
- Learn symbolic transition probabilities at both token granularity (Day 46 Pack 1–2) and string granularity (Day 47 Pack 2), with separate TE tracking for seen and held-out material.
- Replay difficult symbolic structure offline during sleep, measurably reducing held-out prediction error across consolidation cycles (Day 46 Pack 5, Day 47 Pack 4).
- Measure the retention effect of sleep replay via pre/post snapshot methodology (Day 46 Pack 6, Day 47 Pack 5).
- Track cross-modal coupling between structurally independent encoding layers (Day 47 Pack 6).
- Accumulate consolidation statistics across multiple sleep cycles and track long-run trends in held-out accuracy (Day 47 Pack 7).
- Measure transfer across vocabulary tiers at whole-string, character-bigram, and HV-structure granularities (Day 47 Packs 8–11).
- Form predictive HV transformation operators that generalize across the tier boundary without explicit per-string training (Day 47 Pack 11).

### **What the system cannot do**

The architecture still does not:

- Understand code semantically. String and character HVs are constructed from surface form without any semantic annotation.
- Generalize abstract syntax robustly. Character-level transfer ratio = 1.001 (marginal). Whole-string transfer is absent.
- Handle long-range dependencies. All predictors operate on single transitions ( $\text{prev} \rightarrow \text{curr}$ ). Multi-hop sequences are not represented.
- Scale to vocabulary sizes beyond the current 3–6 string curricula without the whole-string predictor growing proportionally.
- Perform hierarchical parsing. The architecture is a flat transition learner with no syntactic hierarchy.

### The structural finding

The most significant result of Days 46–47 is the VSA binding operator (Day 47 Pack 11):  $\text{vsa\_pred\_sim\_ema} = 0.717$  against a random baseline of 0.5. This result is not trivial. The binding operator accumulates XOR vectors from all observed transitions and takes their majority vote. The fact that this operator predicts unseen string transitions with 72% similarity means that the HV space has a consistent directional structure corresponding to the keyword substitution operation "if  $\leftrightarrow$  while". The tier-1 and tier-2 strings differ primarily in this substitution, and the HV encoder -- designed with no knowledge of this structure -- happens to encode it as a consistent vector direction. This is an emergent structural property of the position-weighted bigram encoding, not a designed feature.

This result suggests that the encoding geometry could support associative retrieval of structurally related strings without explicit per-pair training -- a capability that whole-string and character-bigram predictors cannot provide.

---

## 5. Limitations

**Argmax collapse.** After six sleep consolidation cycles (312 replay events, 5,000 ticks),  $P(H|S2) \approx 0.25$  -- above zero, but the predictor argmax continues to select non-H as the most likely continuation. The distribution is improving; the winner is not changing. A larger curriculum or longer run would be required to move the argmax.

**Whole-string predictor does not generalize.** Transfer ratio  $\approx 1.0$  (Pack 8) or 0.305 cold-faster (Pack 9). The whole-string key predictor treats novel strings as entirely new regardless of character-level similarity. There is no associative lookup by structure -- only by identity.

**Character-level transfer is marginal.** Transfer ratio = 1.001 (Pack 10). The advantage from shared character bigrams exists but saturates within a few dozen string presentations. Shared surface form produces minimal functional generalization.

**No semantic grounding.** "if  $x == 1$ " and "if  $x == 0$ " have nearly identical HVs ( $\text{sim} \approx 0.85$ ) despite carrying opposite truth conditions. Semantic content is not represented.

**No hierarchical structure.** The binding operator captures a global average transformation across all string pairs. Per-context operators (one per source string or source cluster) would require  $O(N)$  storage and are not yet implemented. A single global operator overfits to the most common transformation in the corpus.

---



## 6. Current System Architecture

| Days   | Component                                                                                                                                                                          |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| D38–39 | Predictive processing, conceptual binding, confidence modulation                                                                                                                   |
| D40–45 | Trajectory integration, persistence, contextual gating, task-neural coupling, bidirectional feedback                                                                               |
| D46    | Token-level symbolic learning, three-phase curriculum, sleep replay consolidation, retention measurement, structural benchmark                                                     |
| D47    | String-level VSA encoding, string curriculum, cross-modal alignment, multi-cycle sleep accumulation, tier-2 vocabulary expansion, character-level prediction, VSA binding operator |

**State additions (Days 46–47):** approximately 120 new fields across 22 packs in `ikigai/core/state.py`. All fields gated by `enable_X = False` default. Zero routing, TC, or TE drift confirmed for every pack. All injections are in `ikigai.py` before `time.sleep(0.010)`. No permanent modifications to existing logic in `ikigai.py`; all changes are additive.

**Cumulative experiment results:** Days 40–47 total: 439 + 275+ = 714+ PASS, 0 FAIL. All packs deterministic. All OFF paths bit-identical.

---

## 7. Next: Day 48

The VSA binding operator result (Pack 11 `sim_ema` = 0.717) identifies the next productive direction: associative retrieval by structural similarity rather than exact key match.

**Pack 1 -- Per-Context Binding Operators:** Instead of one global binding op, maintain one `vsa_binding_op` per source string key. Prediction for a given `prev_hv` uses the op accumulated from transitions departing that specific string. Expected improvement: `seen_sim` → 0.85+ (tighter per-context distribution). Required: dict of op accumulators, one per unique prev HV seen.

**Pack 2 -- Nearest-Neighbor Retrieval via HV Similarity:** Given a novel string HV (not in the transition dictionary), find the nearest tier-1 neighbor by cosine similarity and inherit that neighbor's transition distribution. This is the first instance of generalization-by-similarity: a novel string would benefit from any knowledge about structurally related strings it has never appeared adjacent to. Required: a stored index of (string\_hv, string\_key) pairs; linear scan at prediction time. Expected: cold-start TE for tier-2 strings decreases significantly relative to whole-string predictor cold starts.

---

## 8. Conclusion

Days 46–47 did not create language understanding.

They created a minimal symbolic prediction architecture: deterministic encoding at token and string granularity, curriculum learning with held-out evaluation, sleep-gated replay that measurably reduces prediction error, multi-cycle consolidation accounting, and the first predictive hypervector transformation operator demonstrating structural generalization at 72% similarity above a 50% baseline.

The system is now operating at the boundary between pure neuroscience simulation and primitive symbolic sequence modeling. The next architectural challenge is generalization-by-similarity: extending learned knowledge from seen structures to novel structures that share geometric relationships in HV space without requiring identical surface forms.

Status: Days 46–47 complete. System ready for Day 48 (per-context binding operators and nearest-neighbor associative retrieval).